



ordinal

Dados Categóricos no Pandas: Categorical, Categorias Ordenadas e pd.cut

O Que São Dados Categóricos?

Conceito Fundamental

Dados categóricos pertencem a um conjunto finito de categorias possíveis. Ao invés de representar um contínuo numérico, eles indicam rótulos ou grupos específicos.

Em pandas, o tipo `Categorical` traz semântica explícita de categorias e é mais eficiente que strings repetidas, economizando memória e processamento.

Exemplo Básico

```
import pandas as pd
cores = pd.Categorical([
    "vermelho",
    "azul",
    "verde",
    "azul"
])
cores
```

❏ **Saída:** ['vermelho', 'azul', 'verde', 'azul']
Categories (3, object): ['azul', 'verde', 'vermelho']

Estrutura Interna: Códigos e Categorias

Códigos Inteiros

Internamente, um `Categorical` armazena valores como códigos inteiros compactos, economizando memória quando há repetições.

```
cores.codes
```

📄 **Saída:** `array([2, 0, 1, 0], dtype=int8)`

Lista de Categorias

As categorias reais ficam armazenadas separadamente no atributo `categories`, explicitando o universo completo de valores possíveis.

```
cores.categories
```

📄 **Saída:** `Index(['azul', 'verde', 'vermelho'], dtype='object')`

Categorias Não Ordenadas vs. Ordenadas

Não Ordenadas

Categorias sem hierarquia lógica — como cores ou tipos de produto. Não há relação de "maior" ou "menor" entre elas.

Ordenadas

Categorias com ordem lógica definida, como níveis de satisfação (baixo < médio < alto). Permitem comparações e ordenação corretas.

Em pandas, definimos a ordem usando o parâmetro `ordered=True` ao criar o categórico:

```
nivel = pd.Categorical(  
    ["médio", "baixo", "alto"],  
    categories=["baixo", "médio", "alto"],  
    ordered=True  
)  
nivel
```

❏ **Saída:** ['médio', 'baixo', 'alto']
Categories (3, object): ['baixo' < 'médio' < 'alto']

Criando Categoria Ordenada: Níveis de Dor

Valores numéricos frequentemente representam níveis discretos em escalas. Transformá-los em categorias ordenadas explicita a hierarquia e melhora a análise.

Código

```
pain = [0, 3, 2, 2, 1]

fpain = pd.Categorical(
    pain,
    categories=[0, 1, 2, 3],
    ordered=True
)

fpain
```

Resultado

Saída:

[0, 3, 2, 2, 1]

Categories (4, int64): [0 < 1 < 2 < 3]

A ordem fica explícita pelo símbolo < entre as categorias.

Adicionando Rótulos Descritivos

Níveis Numéricos

Valores de 0 a 3 representando intensidade

Mapeamento

Dicionário que converte números em textos descritivos

Categorias Textuais

Resultado final com ordem lógica e rótulos intuitivos

```
pain = [0, 3, 2, 2, 1]
mapa = {0: "sem", 1: "baixa", 2: "média", 3: "alta"}
pain_txt = [mapa[x] for x in pain]

fpain_lbl = pd.Categorical(
    pain_txt,
    categories=["sem", "baixa", "média", "alta"],
    ordered=True
)
fpain_lbl
```

❏ **Saída:** ['sem', 'alta', 'média', 'média', 'baixa']
Categories (4, object): ['sem' < 'baixa' < 'média' < 'alta']

Caso Prático: Classificação por Altura

Classificação de pessoas em categorias de altura com ordem lógica: **baixa**, **média** e **alta**.

Dados de Entrada

```
import numpy as np

weight = np.array([
    60, 72, 57, 90, 95, 72
])

height = np.array([
    1.75, 1.80, 1.65,
    1.90, 1.74, 1.91
])

subject = np.array([
    "A", "B", "C",
    "D", "E", "F"
])
```

Vetor de Alturas

height

📄 **Saída:**
array([1.75, 1.8, 1.65, 1.9, 1.74, 1.91])

Seis medidas em metros a serem categorizadas.

Método 1: Lógica Condicional

Loop com Condições

Iteramos sobre cada altura aplicando regras if-elif-else para determinar a categoria.

Testes de Limites

Altura < 1,70 m = baixa; de 1,70 m até menos de 1,90 m = média; a partir de 1,90 m = alta.

Geração de Rótulos

Cada teste resulta em um rótulo textual adicionado à lista de categorias.

```
labels = []  
for h in height:  
    if h < 1.7:  
        labels.append("baixa")  
    elif h < 1.9:  
        labels.append("média")  
    else:  
        labels.append("alta")  
labels
```

📄 **Saída:** ['média', 'média', 'baixa', 'alta', 'média', 'alta']

Convertendo em Categoria Ordenada

Após gerar os rótulos textuais, transformamos a lista em um objeto `Categorical` ordenado, permitindo comparações corretas e análises mais sofisticadas.

```
height_cat = pd.Categorical(  
    labels,  
    categories=["baixa", "média", "alta"],  
    ordered=True  
)  
height_cat
```

❏ **Saída:** ['média', 'média', 'baixa', 'alta', 'média', 'alta']
Categories (3, object): ['baixa' < 'média' < 'alta']

Método 2: pd.cut() – Cortes por Intervalos

Vetorizado

Evita loops explícitos, processando todos os valores de uma vez com alta performance.

Ajustável

Pontos de corte são facilmente modificáveis através do parâmetro `bins`.

Automático

O resultado já nasce como objeto categórico ordenado, pronto para análise.

```
height_cat2 = pd.cut(
    height,
    bins=[0, 1.7, 1.9, np.inf],
    labels=["baixa", "média", "alta"],
    right=False # intervalos [a, b), iguais ao if/elif
)
height_cat2
```

❏ **Saída:** ['média', 'média', 'baixa', 'alta', 'média', 'alta']
Categories (3, object): ['baixa' < 'média' < 'alta']

Por padrão, `pd.cut` usa intervalos `(a, b]`; `right=False` muda para `[a, b)`.

Vantagens do Método por Intervalos

Performance Superior

Ideal para grandes volumes de dados, operando de forma vetorizada sem loops Python explícitos.

Flexibilidade

Os pontos de corte (bins) são facilmente ajustáveis, permitindo experimentação rápida com diferentes classificações.

Resultado Direto

O output já é um objeto categórico ordenado, eliminando etapas adicionais de conversão.

Contagem de Valores

```
pd.Series(height_cat2).value_counts()
```



Saída:

média 3

alta 2

baixa 1

Name: count, dtype: int64

Fluxo de Trabalho Real: Conversão e Inspeção

Na prática, dados frequentemente já existem como textos em DataFrames. Podemos convertê-los facilmente para categorias e inspecionar sua estrutura interna.

```
import pandas as pd
s = pd.Series([
    "baixa",
    "média",
    "alta",
    "média"
])
s_cat = s.astype("category")
s_cat
```

Saída da Série:

```
0    baixa
1    média
2     alta
3    média
dtype: category
Categories (3, object): ['alta', 'baixa', 'média']
```

Esta inspeção revela que categoria não é apenas string — possui estrutura otimizada e semântica própria.

Inspeção Interna

```
# Categorias disponíveis
s_cat.cat.categories
```

```
Index(['alta', 'baixa', 'média'], dtype='object')
```

```
# Códigos numéricos internos
s_cat.cat.codes
```

```
0    1
1    2
2    0
3    2
dtype: int8
```

Sem a lista `categories`, a ordem é alfabética ('alta' < 'baixa' < 'média'). Para a ordem lógica, use `pd.CategoricalDtype(["baixa", "média", "alta"], ordered=True)`.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media. Introdução essencial aos fundamentos de programação e pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media. Guia completo das ferramentas essenciais para análise de dados em Python, incluindo NumPy, Pandas e bibliotecas de visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media. Aprenda os fundamentos de ciência de dados construindo algoritmos e ferramentas do zero com Python.